



Leveraging large language models for BIM-based automated compliance checking

Odin Iversen^{ID}, Lizhen Huang^{ID}*

Norwegian University of Science and Technology (NTNU), Department of Manufacturing and Civil Engineering, Teknologivgen 22, Gjøvik, 2815, Norway

ARTICLE INFO

Keywords:

Automated compliance checking
Digital building permitting
Large language models
Building information modeling

ABSTRACT

Current methods of checking regulatory compliance in the architecture, engineering, construction, and operations (AECO) industry are mostly manual, time consuming and error prone. This paper, using design science research (DSR), proposes an artifact that leverages a large language model (LLM) for automated compliance checking (ACC) to directly interpret regulations, extract BIM data, execute checks, and generate detailed reports. For rule interpretation, the artifact achieves high F1-scores (97% for classification, 100% for dependency identification). For building model preparation, it correctly selected data extraction tools in 97% of cases. In rule execution, it demonstrated 97.7% accuracy and significantly outperformed a naive baseline, which highlights the need for a structured framework. Finally, the artifact generated detailed reports that included the LLM's reasoning. The key finding is that an LLM-based reasoning engine enables a holistic approach that overcomes the manual rule digitization bottleneck in traditional ACC systems.

1. Introduction

Compliance checking against complex regulations remains a significant bottleneck in the architecture, engineering, construction, and operations (AECO) industry, leading to costly delays and errors [1]. The sheer number of rules, as well as the interdisciplinary nature of the regulations, make compliance checking challenging and time-consuming. Technologies such as building information modeling (BIM) are expected to facilitate digital building permitting (DBP) through automated compliance checking (ACC) [2]. However, although ACC is expected to have a significant economic impact on the AECO industry and is a heavily researched topic [2], it has not seen meaningful adoption in real-world practice. This is often attributed to significant hurdles such as the lack of shared open standards for representing regulation clauses digitally and, until recently, limitations in artificial intelligence (AI) technologies for interpreting regulatory text [3].

One of the main challenges facing ACC is addressing the first class of functionality: **rule interpretation**, which involves understanding the semantic meaning and requirements embedded within the regulatory text [1]. In existing literature, the primary approach to interpreting rules is through the digitization of regulations, converting them into a **machine-interpretable format**. However, with the advent of large language models (LLMs), machines are becoming increasingly capable of directly interpreting natural language. This shift presents an opportunity to explore alternative approaches to rule interpretation that

leverage the LLMs' direct understanding to omit the labor-intensive and error-prone process of manually coding each regulation into logic rules. In light of this, the present study addresses the following research question: **To what extent can large language models facilitate BIM based automatic compliance checking of natural language building regulations?**

This research aims to contribute by (1) evaluating the feasibility of LLMs for end-to-end ACC tasks on natural language regulations, (2) providing an agentic artifact based on DSR principles, and (3) offering insights into the strengths and limitations of LLMs in this specific application context. To further specify and structure the investigation, the research question is broken down into four sub-questions, each addressing one of the core ACC functionalities:

- **Rule Interpretation:** To what extent can LLMs contribute to rule interpretation?
- **Building Model Preparation:** To what extent can LLMs assist in extracting information from BIM models for compliance checking?
- **Rule Execution:** To what extent can LLMs execute compliance checking based on interpreted rules and model data?
- **Reporting Checking Results:** To what extent can LLMs help create detailed compliance reports from the checking results?

To explore these questions, the design science research (DSR) methodology is employed [4]. Consistent with DSR principles, which

* Corresponding author.

E-mail address: lizhen.huang@ntnu.no (L. Huang).

Nomenclature

ACC	automated compliance checking
AECO	architecture, engineering, construction, and operations
AI	artificial intelligence
BIM	building information modeling
DBP	digital building permitting
DiBK	Norwegian Directorate for Building Quality
DSR	design science research
FEA	finite element analysis
IFC	Industry Foundation Classes
IDS	Information Delivery Specification
JSON	JavaScript Object Notation
LLM	large language model
NLP	natural language processing
OWL	Web Ontology Language
POS	part-of-speech
RAG	retrieval-augmented generation
SNACC	semantic NLP-based automated compliance checking
SPARQL	SPARQL Protocol and RDF Query Language
TEK17	Norwegian Building Regulations (Byggt teknisk Forskrift, 2017)
UML	Unified Modeling Language

focus on creating and evaluating innovative artifacts to solve real-world problems, this research develops and evaluates an agentic artifact capable of autonomous compliance checking. Rigor and relevance is ensured by applying knowledge about theoretical foundations and existing methodologies from the knowledge base, and by identifying business needs through engagement with stakeholders. **Several meetings were conducted with the Norwegian Directorate for Building Quality (DiBK) and Arkitektum (a Norwegian company specializing in digital solutions for planning, construction, geodata, and archiving) to ensure applicability in the appropriate environment.** This collaboration helped ascertain the state of DBP and ACC in Norway and identify Norwegian Building Regulations (Byggt teknisk Forskrift, 2017) (TEK17) §12-14 Stairs as a suitable regulation for LLM-based ACC, due to ongoing digitization efforts and its predominantly geometric nature. The development and evaluation of this artifact aims to contribute to the knowledge base and facilitate practical application [4, p. 80]. The remainder of this paper is organized as follows: Section 2 reviews related work in automated compliance checking. Section 3 introduces the core techniques for building LLM-based agents, which form the conceptual foundation for our work. Section 4 details the DSR methodology and the design of the proposed agentic artifact. Section 5 presents the artifact's evaluation, followed by a discussion of the findings in Section 6. Finally, Section 7 concludes the paper and suggests directions for future work.

2. Related work in automated compliance checking

Automated compliance checking can be divided into four classes of functionality: rule interpretation, building model preparation, rule execution and reporting checking results [1]. There is limited research on applying LLMs in ACC. Therefore, this section first aims to give a general overview of existing work on the development of each of the four classes of functionality, before delving deeper into a final subsection on how LLMs have contributed to ACC, as well as some examples of real world implementation of ACC systems.

2.1. Rule interpretation

The first class of functionality of ACC is rule interpretation. Current methods of rule interpretation in the context of ACC are centered around digitizing the rules into a format interpretable for traditional computers, either through translation or extraction [1,5–12]. The machine-readable rules can be captured using three main approaches: (1) **domain-specific schemas like Industry Foundation Classes (IFC) and Information Delivery Specification (IDS)**; (2) **general data standards like JSON**; and (3) **logic-based and Semantic Web formalisms like First-Order Logic, Web Ontology Language (OWL), and SPARQL Protocol and RDF Query Language (SPARQL)** [13]. The suitability of regulations in terms of digitization varies, and Zhang and El-Gohary [14] proposed features that could be used to calculate the complexity and computability of building codes.

Buildings are comprised of numerous interconnected systems, each governed by complex regulations. Consequently, interpreting these regulations for ACC requires a range of approaches, as illustrated in Fig. 1. While these methods are often used in **combination** (for instance, powerful deep learning techniques are frequently applied to the field of natural language processing (NLP)), we discuss them here according to the primary focus of different streams of literature. The interpretation process itself may therefore involve manual methods, traditional NLP, deep learning, or combinations thereof. Common outcomes of this process include directly executable rules, an ontology, or a knowledge graph. Additionally, an ontology can either directly generate executable rules or serve as the foundation for the knowledge graph, which in turn can also be used to produce the executable rules. Thus, the outcome could be a combination of various data structures and formats, with the end result and goal being to create rules to be executed as the third class of functionality of ACC.

To illustrate the diverse approaches to rule interpretation developed by researchers, this section draws on several examples from existing literature, that show a progression from manual to more integrated methods. Kincelova et al. [15] aimed to demonstrate the feasibility of ACC for provisory and dependent clauses within fire safety regulations. Their method involved manually interpreting these regulations to extract the necessary concepts and attributes, ultimately showing such ACC was possible. While such manual methods are effective, they are also labor-intensive. Seeking greater efficiency for specific data formats, Xue and Zhang [9] proposed a more efficient approach for extracting regulatory requirements from regulations containing tables, aiming to improve data coverage for ACC. They addressed this by developing a method for semi-automated table layout detection, which converts the requirements into a machine interpretable database. Moving from specific extraction challenges to more foundational issues of rule representation, Zhang et al. [16] aimed at facilitating rule representation in healthcare building compliance. They provided a foundational checklist for ACC development based on the systematically identified capabilities. Building on such foundational work, Most recently, Patlakas et al. [17] sought to enhance ACC by integrating finite element analysis (FEA). Their holistic approach involved proposing a top-level ontology, centered around modeling statements from building regulations, to facilitate the incorporation of FEA data within the IFC schema for compliance verification. Taken together, these examples illustrate a clear trajectory in the field: a move away from isolated, manual interpretation toward more holistic, semi-automated, and ontology-driven frameworks.

NLP has traditionally been a key component of rule interpretation. Zhang and El-Gohary [18] proposed a unified ACC system called semantic NLP-based automated compliance checking (SNACC) using semantic information elements from Zhang and El-Gohary [19]. The system used NLP for automatic transformation of regulations into logic rules, algorithms for automatically extracting information from IFC-models, and logic reasoning algorithms. Building on this work, Zhang [20] showed that a tree-based visualization method of the

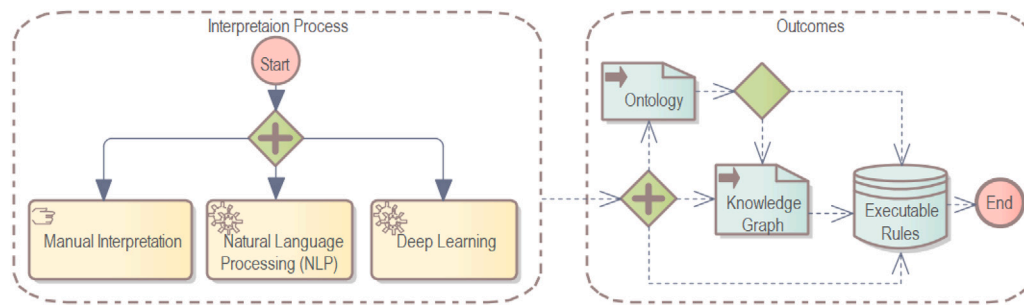


Fig. 1. Conceptual overview of rule interpretation approaches. Notation (BPMN 2.0): hand = Manual Task; gear = Service Task.

regulation analyzed using SNACC could significantly improve readability and understandability of logic clauses. Whereas the SNACC system focused on a complete transformation pipeline, another key approach involves the preliminary classification of rules. Nawari [6] proposed a framework for semi-automated compliance checking, where the first phase consisted of classifying rules into provisory (conditional), content, ambiguous and dependent clauses. The classification of the rules is performed by the transformation reasoning algorithm with a special syntax for digitizing the regulations. Taken together, these systems demonstrate how traditional NLP was used to create structured frameworks for semi-automating rule interpretation.

Deep learning has also been used to facilitate rule interpretation and digitization, which is made possible by labeled training data. Consequently, a significant research effort has focused on creating these foundational resources. Hettiarachchi et al. [21] introduced the CODE-ACCORD manually annotated dataset, intended to support task such as text classification, relation extraction and entity recognition. Similar work has been done in Vienna, Austria by Recski et al. [22] who presented the BRISE-Plandok. Building on such datasets, researchers have proposed various models to automate specific interpretation tasks. Zhang and El-Gohary [23] proposed an alternative representation, suggesting that the traditional natural language text of building codes be replaced with “intelligent building codes”. These codes are highly structured and computer-understandable, specifically aiming to bypass the error-prone process of information extraction. They also proposed a deep-learning-based method to automatically generate natural language requirements from this formal representation. This conversion is facilitated by their related work on a new hierarchical representation of requirements [8], which uses a deep learning method to automatically extract semantic relationships between words in building codes, transforming them into semantic computable requirements. Other models have focused on more granular tasks, such as the deep learning-based model for joint extraction of entities in form of triplets proposed by Li et al. [7]. Along these lines, Xue and Zhang [24] attempted to improve the performance on part-of-speech (POS) taggers on building codes by using error-driven transformational rules to revise results. To further improve performance, Xue and Zhang [25] proposed a deep learning based POS tagger specialized on building codes. This was designed to facilitate automated rule conversion, as the syntactic information provided by POS tagging is a crucial first step for accurately extracting requirements and relationships from the text. These studies demonstrate that while deep learning can effectively tackle discrete interpretation sub-tasks, its application is dependent on the prior availability of large, domain-specific training corpora.

Ontologies are frequently used to support the entire ACC process, often serving as the foundation for generating executable rules. Early research established their utility, with work such as Zhou and El-Gohary's [12] proposal for an ontology-based algorithm to perform information extraction and digitize energy requirements. Lee et al. [26] proposed a high-level approach for rule interpretation where requirements were first split into translated atomic sentences. From these sentences, they derived methods based on the verb phrases to create a new,

high-level representation of the regulations, which they showed could be implemented as executable KBimCode. Meanwhile, Jiang et al. [27] demonstrated that multiple ontologies could be combined, serving as knowledge bases, to facilitate ACC. Building on these foundations, other research has focused on improving the richness of the semantic representation. Conventional ontology-based approaches often struggle to represent relationship constraints and distinguish between checking conditions and requirements. Addressing these challenges, Li et al. [28] proposed a semantic representation method that explicitly integrates both relationship and spatial constraints. Their approach introduces a structured code representation model and establishes mappings between a building information ontology and building code concepts. While the work by Li et al. focused on enriching the semantic representation itself, a parallel trend involves integrating such representations into more comprehensive, multi-component frameworks that combine ontologies with other AI techniques. Building on the existing SNACC framework, Wu et al. [29] proposed I-SNACC, which focused on improving the building information extraction and representation phases using invariant signatures and logic reasoning to enable automatic ontology-based rule generation. In a similar vein, Li et al. [30] proposed an integrative conceptual framework for BIM-based ACC by constructing an ontology and automatically labeling and adding logical markers to the building standard using deep learning. Dynamo and RevitAPI was used for parameter addition/extraction. Finally, a matching method was devised for matching the data from the BIM model with the knowledge graph triples to perform ACC. This progression highlights a clear trend: using ontologies not just as data models, but as the central semantic backbone for increasingly automated, multi-component ACC systems.

Closely related to ontologies, knowledge graphs have also been used in existing research on rule interpretation for ACC. For example, Jiang et al. [31] proposed a multi-scale modeling approach to support knowledge representation of building codes, incorporating conceptual, logical, and correlational representations to facilitate ACC. The approach enhanced semantic searching, knowledge recommendation, and improved the transparency and efficiency of compliance-checking processes. However, the rule interpretation and conversion into the clause-entity knowledge graph still relied on domain experts. Seeking to address this manual bottleneck, Kruiper et al. [32] developed a semantic search engine to identify and extract relevant requirements from a large corpus of building documents for compliance checking of building designs. An automatically generated knowledge graph was used to expand user queries to improve search results in cases where similar terminology is used. Notably, they argue that complete semantic parsing of building regulations is impossible due to many clauses requiring natural language understanding, thus making end-to-end ACC unfeasible. This ongoing debate about the limits of semantic parsing highlights the need for new paradigms, such as the direct interpretation model proposed in this paper.

2.2. Building model preparation

The second class of functionality of ACC is building model preparation. To execute the digitized rules created during rule interpretation, the building model must contain sufficient and retrievable information, that is possible to align with the regulatory concepts.

Therefore the first challenging part of building model preparation is **aligning regulatory concepts with BIM concepts**. A significant stream of research has focused on semi-automating this alignment. For example, an early approach by J. Zhang and El-Gohary [33] **proposed a semi-automated method for extending the IFC schema by using NLP to extract regulatory concepts from text and match them to existing IFC concepts**. More recently, research has leveraged modern deep learning models for this task. R. Zhang and El-Gohary [34] proposed a transformer-based deep learning model for aligning the semantics of the IFC format with the semantics of natural language regulations. The results indicated that adapting pre-trained models with domain specific data is important, and that injecting contextual information and incorporating natural language definitions of IFC concepts could improve results. A different method was proposed by Wu and Zhang [35] for BIM-model validation based on building code requirements. A key feature of their framework was a module for developers to manually refine the automatically generated logic rules, a step that was required to achieve 100% accuracy on their training data. The method can be used to extend model information to match concepts from building codes based on invariant signatures from IFC models, and to generate logic facts to store the extended information. These studies show a clear progression of techniques for semantic alignment, but also underscore the persistent challenge of achieving full automation without requiring manual refinement.

The next challenge is **to ensure sufficient information within the building model** to facilitate rule execution. Research has tackled this challenge from several angles, primarily by either enriching the model with new data or by standardizing how that data is defined and retrieved. One approach is to manually supplement the missing information needed to perform ACC [36]. A more automated approach is semantic enrichment, which uses techniques such as rule-based inferencing, geometric calculations, and AI to automate the extensive manual preprocessing that input models typically require. To better structure this task, Bloch and Sacks [36] used a clustering process to create a hierarchical classification of the different types of semantic enrichment needed for code compliance. An example of a work that performs such enrichment is [37], which developed a method for automatically populating BIM models with semantic data derived from implicit geometric and topological information within the model itself, making the models more suitable for compliance checking. In contrast to enrichment, other work has focused on standardizing information requirements. Nawari's [38] proposed framework uses a traditional feature extraction algorithm for objective data, combined with fuzzy logic to handle ambiguous regulatory requirements when evaluating them against data from the BIM model. The information extraction is facilitated by developing an information delivery manual providing exchange models that can be used to make a model view definition. Thus each discipline can focus on their relevant subset of the IFC-schema. Following this focus on standardization, another approach to ensuring sufficient information within the building model is to improve the retrieval of information that is implicitly stored within the BIM models. Fischer et al. [39] extended the IDS schema to allow for the retrieval of properties of related objects from IFC models. This helped facilitate escape route analysis through properly defining information requirements, and showed promise for automatic compliance checking. However, the approach must be combined with other checking logic for more complex regulations. Collectively, these diverse strategies highlight the persistent challenge of ensuring a BIM model is informationally complete for a given compliance task.

Other recent work has focused on LLM-based assistants for general information retrieval from BIM models. For example, Zheng and Fischer [40] developed a framework for interpreting natural language queries, while Fernandes et al. [41] proposed a digital assistant for real-time interaction with BIM. While promising for general design assistance, these approaches are not tailored for the specific demands of ACC, as they do not address the extraction of implicit geometric and spatial information often required to verify compliance with complex regulatory texts.

2.3. Rule execution

Rule execution, the third class of ACC functionality, has traditionally been approached through three main paradigms: hard-coded programs, logic-based reasoners, and data-driven machine learning models.

The most conventional approach involves a fully fledged executable program. The program is usually hard coded with the interpretation of the building codes, and explicitly extracts the relevant information from the building model. One example is Jeongmin and Shin [42] who implemented a selection of Korean building codes using Dynamo in Revit. They showed that automatic rule execution could be performed on building codes regulating open spaces beyond the building itself. However, a significant limitation of this method is that the applicable rules were manually selected and digitized, and the building model was manually prepared.

A more flexible approach involves the use of logic-based reasoners. In this paradigm, regulations are converted into a formal logic representation, and a general-purpose inference engine is used to check compliance. A prominent example is the previously mentioned SNACC system [19], which automatically transforms regulatory information into logic rules to prepare for automated reasoning.

Alternatively, when large amounts of data are available, machine learning-based approaches can be used for rule execution. This process involves first training a model, such as a neural network or an ensemble method, on example data. Once trained, the model can check new cases and execute the rules it has learned, either through pattern recognition and feature extraction or through decision-based logic. One example is Li et al. [43], who used a random forest algorithm with particle swarm optimization, fuzzy set logic, and Dempster-Shafer fusion theory to identify geological conditions for automatic compliance checking in tunnel boring machine tunneling.

These three paradigms highlight a fundamental division in rule execution methodologies: one relies on explicitly programming human-interpreted logic (either directly or via a reasoner), while the other infers logic from data. All, however, are dependent on the prior, labor-intensive digitization of rules.

2.4. Reporting checking results

The fourth and final class of functionality of ACC is the reporting of checking results. However, most ACC research primarily focuses on rule interpretation and building model preparation, while the process of reporting checking results is often overlooked or not described in significant detail. Existing systems have employed a range of reporting methods, from graphical visualization to structured data and simple textual outputs. The LicA system, for example, reported checking results through various methods, including visualization in its custom graphical user interface, named LiCAD. In addition, the results could be reported in a natural language based table overview, containing short descriptions of checks alongside the severity and results. Finally, the model with checking results could be exported as an XML file [44].

Other systems have also relied on tabular or simple pass/fail statements. Nawari [6] reported the checking results as a table showing identifying information for each checked building component, as well as whether or not it passed or if the results were not available. Macit Ilal

and Günaydin [45] reported checking results as pass or fail statements inside the compliance checking application, alongside identification codes for objects and rules. More detailed textual descriptions are less common, though BIMTRAC3 gives a textual description, stating compliance results, regulation text and information extracted from the BIM-model [46]. Overall, while varied, these reporting methods often lack the detailed, human-readable reasoning that modern LLMs can generate, representing a clear opportunity for improvement in the field.

2.5. LLMs in ACC

While some existing research has explored LLMs applications in ACC, its primary focus has been on rule interpretation, particularly in the context of digitizing regulations. Several studies have demonstrated this approach using different target formats. Fuchs et al. [11] showed how intermediate representations could improve semantic parsing of building regulations when converting rules into LegalRuleML using transformer based semantic parsing. Statements were manually classified by domain experts into prescriptive statements and constitutive statements. The method was then expanded and improved upon by Fuchs et al. [47] by incorporating contextualization, chain-of-thought prompting and self consistency. Targeting a different structured format, Al-Turki et al. [48] used LLMs for digitizing building codes into YAML, based on the Requirement, Applicability, Selection, and Exception (RASE) methodology. The models ability to interpret regulations was demonstrated with preliminary few-shot prompting, and further enhanced with fine-tuning and progressive active learning. Moving from structured data to executable code, Yang and Zhang [49] proposed a framework for supporting developers in converting natural language regulations into logic programming scripts for use in ACC. The current state of the framework shows that the LLM-based translator is able to accurately generate logic rules in some cases. However, a human in the loop is still needed for manually generating and evaluating the logic code, and the framework is intended to support the translation process, not to fully automate it. Similarly, Chen et al. [50] applied LLMs to rule interpretation, but their focus remained on the traditional digitization paradigm; the LLMs was used to extract and classify information, outputting JSON that was then converted into SPARQL queries.

A notable exception that moves beyond digitization assistance is the work by Ying and Sacks [51], who proposed an LLM-based agentic framework for querying requirements and performing compliance checks using a ‘reason and act’ approach. However, their evaluation was limited to a single requirement and noted issues with inconsistent LLMs reasoning and correctness, while providing limited detail on how the framework addresses the four distinct stages of ACC. In contrast, our work provides a more granular breakdown of each ACC stage and presents both qualitative and quantitative results from a significantly larger and more diverse set of regulations.

2.6. Real world implementation

Several real-world ACC systems have been developed, typically tailored to specific national regulations and use cases. For example, Kim et al. [52] developed a prototypical KoreanBIM e-submission framework based on openBIM and IFC as part of an ongoing government initiative to improve the national SEUMTER system. In Turkey, Aydın [46] developed BIMTRAC3, a web-based ACC software using BIM for municipal authorities. The study demonstrated that BIMTRAC3 significantly reduces the time required for compliance checks compared to traditional manual methods, improving accuracy and efficiency. Focusing on a specific engineering domain, Martins and Monteiro [44], presented LicA, an application specifically designed for the purpose of performing ACC on building water network designs according to Portuguese regulations; their approach centered on using IFC as the exchange format to ensure interoperability. To address a different type of regulation, Altintas and Ilal [53] proposed a zoning domain model

showing that GIS data can complement BIM data without expanding BIM schemas. These implementations demonstrate the practical benefits of ACC in various contexts, but they also underscore that such systems are often highly specialized, targeting particular national codes or specific compliance domains.

2.7. Summary of related work and identified gaps

Current research in Automated Compliance Checking highlights significant advancements across its four classes of functionality – rule interpretation, building model preparation, rule execution, and reporting – yet also reveals persistent challenges. Rule interpretation efforts have predominantly focused on digitizing regulations into various machine-readable formats, employing NLP, deep learning, ontologies, and knowledge graphs; however, achieving full, robust automation in converting complex, nuanced legal text remains a challenge, often requiring substantial manual input or semi-automated processes. Similarly, building model preparation grapples with aligning regulatory semantics with BIM data and ensuring informational completeness, with current strategies ranging from manual enrichment to developing standardized data views. While rule execution mechanisms exist, and reporting methods vary, these later stages often receive less detailed attention in the literature compared to the complexities of rule interpretation and model preparation.

The recent introduction of LLMs into ACC shows considerable promise, particularly in supporting rule interpretation. However, a review of current applications reveals two key research gaps that this paper aims to address. First, while LLMs have been used to assist in digitizing regulations, there is a lack of research on end-to-end ACC frameworks that use an LLM’s direct natural language understanding to bypass the digitization bottleneck entirely. Second, there is a corresponding gap in research on using LLM-driven reasoning for the complex task of building information retrieval in the ACC context, particularly for extracting the implicit geometric and spatial information required for compliance checks.

3. Core techniques for building LLM-based agents

Enhancing LLMs performance can be approached in two main ways: modifying the model itself through fine-tuning, or improving the interaction with the pre-trained model. This paper focuses on the latter strategy, where the “user” interacting with the LLMs is the software artifact itself, not a human end-user. This approach is best conceptualized as an LLM-based agent, where the LLM acts as a core reasoning engine to autonomously perform complex tasks. While the preceding literature review highlights various applications of such methods, this section provides a more detailed background on the two foundational techniques that are central to our artifact’s design: retrieval-augmented generation (RAG) and prompt engineering. These techniques serve as the key building blocks for enabling this agentic functionality.

3.1. Grounding agents with external knowledge using RAG

For an LLM-based agent to perform reliably in a specialized domain like ACC, it must be grounded in factual, external knowledge. Retrieval-augmented generation (RAG) is the primary mechanism for providing this grounding, improving AI-generated content by providing the LLMs with additional external knowledge. This is achieved by creating a framework that combines information retrieval the LLM-based content generation process [54]. To facilitate this, source documents are often pre-processed through document splitting or vectorization to enhance searchability. The performance of various document splitting techniques has been evaluated; however, the study concluded that prompt engineering had a more significant impact on achieving high overall accuracy and recall than the specific document splitting technique used [55]. Researchers have also shown that the effectiveness of RAG

can be significantly improved by combining it with other methods, such as Chain of Thought (CoT) prompting in the legal domain [56], or by classifying and decomposing complex queries before retrieval [57].

Several studies have demonstrated successful RAG implementations. For example, Tran et al. [58], who proposed an LLMs based query answering system, while Kalyuzhnaya et al. [59] successfully used a vectorized database with city data to improve performance in urban planning tasks. Other work has focused on using RAG to reduce hallucinations, such as the multi-layered legal query answering system developed by Wahidur et al. [60]. In the agentic artifact developed for this research, the agent leverages RAG as a foundational capability across all stages of the ACC process. It is used to assemble context-rich queries during rule interpretation, retrieve necessary information during building model preparation, and ground the final verdict in rule execution, ensuring that the agent's reasoning is consistently based on both the specific regulatory requirements and the building model data.

3.2. Directing agent reasoning with prompt engineering

If RAG provides the agent with its knowledge, prompt engineering is the primary mechanism for directing its reasoning and actions. Prompt engineering is an emerging discipline focused on designing, developing, and optimizing prompts to enhance the outputs of generative AI models like LLMs [61]. Given the statistical nature of these models, poor inputs naturally lead to poor outputs. Consequently, it is the fundamental technique for programming an agent's workflow, defining its logic for specific tasks, and structuring its final output.

One widely used technique is called Chain of Thought (CoT) prompting, which encourages the agent's underlying LLM to produce longer responses with reasoning steps included in the response. This step-by-step process helps the model allocate computational effort to each stage of reasoning, reducing the likelihood of errors that can occur when attempting to derive the answer in a single step. Hebenstreit et al. [62] tested various CoT-prompts to evaluate enhanced reasoning capabilities. Their findings demonstrated that applying reasoning strategies during prompting outperformed the direct approach without CoT prompting. The most effective prompt for GPT-4 was: "Let's work this out in a step-by-step way to be sure we have the right answer". Similarly, Moghaddam and Honey [63] found that CoT prompting and step-by-step thinking improved LLMs' theory of mind capabilities.

Beyond single-prompt techniques like CoT, prompt engineering can also be used to orchestrate more elaborate strategies that involve multi-step processes or interactions between multiple model instances. One such approach focuses on self-correction through reflection. Shinn et al. [64] proposed a prompt-driven reflection process involving three models: an actor, which generates solutions and interacts with the input; an evaluator, which scores the actor's proposed solution; and a self-reflection model, which analyzes both the solution and its score. The process runs iteratively until a satisfactory score is reached, producing a final output. This approach demonstrated improvements in sequential decision-making, reasoning, and programming benchmarks.

An even more advanced application of this principle is the creation of multi-agent systems, where prompting is used to define distinct roles and interaction protocols for multiple LLM instances. Nair et al. [65] introduced Dialog-Enabled Resolving Agents (DERA) using GPT-4, which outperformed the base GPT-4 model on summarizing medical conversations and generating care plans. By employing two agents engaged in discussion, accuracy was improved in the final output. One agent acted as the "researcher", responsible for processing information and identifying problems, while the other served as the "decider", with autonomy to determine whether to integrate the researcher's inputs. The agents collaboratively discussed initial inputs before generating a final response. This approach enhanced performance on the MedQA benchmark, yielding higher-quality answers and reducing hallucinations. Similarly, Kalyuzhnaya et al. [59] showed that a multi-agent

approach allowed for significant improvements over base LLMs performance in urban planning, and allowed for significant reductions in human decision making time. Response accuracy remained high even when using smaller LLMs. While these advanced multi-agent frameworks are a promising direction, the artifact in this study utilizes a more direct, single-agent pipeline, where different tasks are handled as sequential steps in a workflow defined by the prompts.

Prompt engineering is central to the agentic workflow of our artifact. It is used to define the logic for Rule Interpretation, guides tool selection in Building Model Preparation, ensures accurate Rule Execution, and structures the output for the final Reporting of results. As noted previously, our current artifact utilizes a more direct, single-agent pipeline, but the principles of agentic design inform the overall architecture and point toward future enhancements where distinct agents could handle specific ACC stages.

Collectively, the techniques discussed in this section make it possible to build sophisticated LLM-based agents without requiring modifications to the underlying models. These strategies include sophisticated prompt engineering, which guides the agent's reasoning processes and structures its output, encompassing techniques from Chain of Thought to more complex approaches like multi-step reflection and the development of agentic frameworks involving collaborative LLMs interactions; and RAG, which grounds the agent with external, domain-specific knowledge to improve factual accuracy and relevance. By applying these methods, developers can create agents capable of producing more reliable, accurate, and contextually appropriate responses from LLMs, which is crucial for demanding applications like Automated Compliance Checking.

4. Artifact design and development

To assess to what extent LLMs can facilitate BIM-based ACC, an artifact was developed based on DSR principles. In contrast to the current state of art, the proposed artifact is developed to be LLM-centric, bypassing the need for digitizing regulations into computer code or logic rules. This allows for more direct use of the original natural language clauses as a basis for BIM model preparation, rule execution and compliance report generation, all performed by the LLM-based ACC artifact.

The artifact is implemented in Python and leverages the OpenAI API to access the GPT series of models. These models were selected as they consistently ranked as the top-performing publicly available models on major LLMs benchmarks at the time development began at the start of 2024. Our methodology utilizes these models as-is; no fine-tuning or further training was performed.

Fig. 2 illustrates the artifact's operational workflow, which comprises several key stages. This workflow uses the building code, a custom JSON Schema, and the BIM model as inputs to produce compliance reports as its final output. To achieve this, the artifact first converts the building code into JSON format to segment the data by clause. The resulting JSON is then validated against a proposed JSON Schema to ensure the correct hierarchical structure of the data. Next, the regulation is automatically pre-processed by identifying dependencies between clauses and generating query strings suitable for processing by the LLMs. These generated queries serve as the primary output of the Rule Interpretation stage, and are used as inputs for the Building Model Preparation and Rule Execution stages. Each rule is subsequently classified based on its suitability for BIM-based compliance checking, specifically on whether the requirement includes geometric demands. Metadata from the BIM-model is then used to check which clauses apply to the building. Geometric information is then extracted from the BIM-model based on the LLMs interpretation of the applicable clauses. The extracted information and generated clause queries are then used in combination for rule execution performed by the LLMs. Finally, a detailed compliance report and compliance summary is generated based on the analysis from rule execution.

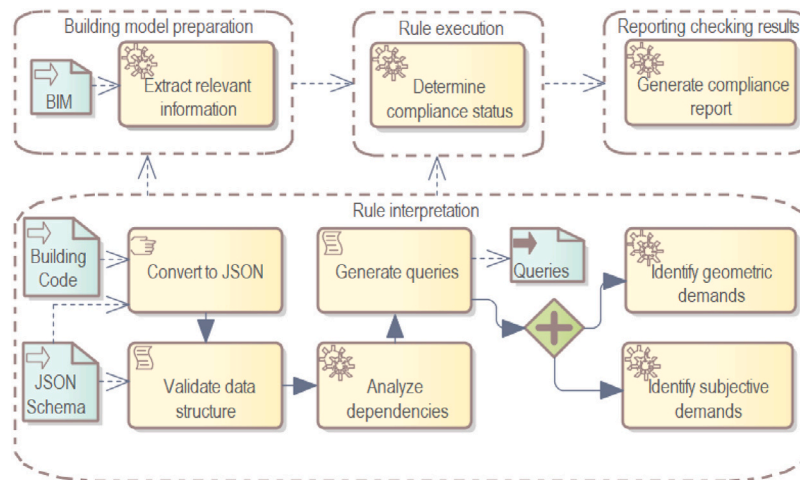


Fig. 2. Workflow of the proposed ACC artifact, organized by four main functionalities (BPMN 2.0). Notation: hand = Manual Task; gear = Service Task (API to LLMs); scroll = Script Task (inline code); hollow arrows = inputs; filled arrows = outputs.

A foundational technique employed throughout this workflow is RAG. At nearly every stage, the artifact retrieves contextual information from an external knowledge base to ground the LLM's reasoning and reduce hallucinations. Our implementation uses input-layer integration [54], where relevant regulatory text is retrieved from the structured JSON. This retrieval is handled by iteratively processing each segment of the regulation, an approach well-suited for the DBP context where full compliance with the entire regulation is required. While this exhaustive method ensures completeness, it is not optimized for the design phase, where future work should focus on incorporating more targeted filtering and search functionality.

4.1. Rule interpretation

Existing research on rule interpretation focuses on converting natural language regulations into a machine-readable and machine-interpretable format through digitization. Many approaches use LLM-based techniques to perform the digitization, showing promising results. This demonstrated capability for digitizing regulations, however, calls into question the assertion that LLMs operate without some form of interpretive understanding.

The argument put forth in this paper is that LLMs can effectively understand natural language regulations, and thus natural language regulations themselves have become machine-interpretable with the advent of LLMs technology. Consequently, the traditional ACC process of interpreting regulations through digitization is being rendered obsolete, and natural language regulations can potentially be used directly in ACC using the methods proposed and as demonstrated by the artifact. This argument puts into question the efficiency of existing methods where LLMs assist in digitizing regulations, as this could introduce added and perhaps unnecessary complexity. To correctly digitize regulations, the model must be capable of understanding both the natural language regulation and the target format. However, if the model can understand the natural language format, that understanding could itself be directly applied in the rule execution, thus negating the necessity of a model capable of understanding the digitized regulation formatting.

Instead of separating the automatic compliance checking process into steps targeting the digitization of regulations, the process can be separated into steps enhancing the direct interpretation of natural language regulations. This is more akin to traditional compliance checking, only automated, where the LLM-based system acts as an agent with a similar role and activities as a human professional.

Due to the inherent ability of LLMs to process and “understand” natural language, the basis for rule interpretation differs from existing studies. The rules are not digitized in the traditional sense of being converted into formal computer-interpretable formats such as code, logic rules, ontologies, or knowledge graphs, as the natural language representation will be used directly by the LLMs, throughout the entire compliance checking process. Nevertheless, some preprocessing is required, as inputting the entire regulation for each discrete operation within the compliance workflow would lead to excessive data overhead and inefficiency. Rule interpretation is therefore handled by segmenting the regulation and combining parts of it into context-rich queries. An example of a generated query is provided in Appendix A.

The first sub-process uses the regulation and the proposed JSON Schema as inputs to convert the regulation into structured JSON (see Listing 1 for a condensed example), primarily by segmenting its clauses as defined by the schema. The schema's structure is visualized as a Unified Modeling Language (UML) diagram in Appendix B.1, with an excerpt from the JSON Schema code in Appendix B.2. The target regulation for this work, TEK17, is available online (translated regulatory text in Appendix D) and was initially processed using iterative regex-based HTML parsing to convert it into a structured format. However, inconsistencies within the HTML code such as the figure names sometimes being above and sometimes below the figure itself made this challenging. Thus, the contents of the resulting JSON had to be manually verified.

Before generating the natural language queries, the segmented regulation text had to be grouped together, ensuring all relevant regulatory text that must be evaluated simultaneously could be provided to the LLMs as a single query. This required the second sub-process of identifying dependent clauses. In the context of DBP, compliance checking requires the building to be compliant with all clauses in the regulation. Therefore, clauses that are closely related are not considered dependent in this context if sequential compliance checking of the two clauses gives the correct answer. Only if two or more clauses must be evaluated simultaneously are they considered dependent, as they must then both be provided together to the LLMs during rule execution. To achieve this, we developed an agentic, multi-step workflow using a sequence of targeted LLM prompts. First, an initial assessment identifies potential dependencies (Appendix C.1.1). If a dependency is suspected, a second prompt extracts and formats the identifier of the target clause (Appendix C.1.2). Finally, after retrieving the target clause's text, a third prompt performs the final evaluation using both clauses together (Appendix C.1.3).

Listing 1: A condensed excerpt from the TEK17 regulation, converted to a structured JSON format according to the proposed schema.

```
{ "chapters": [
  { "number": "$ 12", "title": "Planløsning...", "sections"
    ↪ : [
    { "number": "$ 12-14", "title": "Trapp", "paragraphs": [
      { "number": "$ 12-14. (2)", "letters": [
        { "number": "$ 12-14. (2) b)",
          "rules": [{ "text": "Håndløperen skal følge..."
            ↪ } ] },
          "guidances": [ "...", "..."
            ↪ ], "..."
          } ], "..."
        } ] }
      ] }, "..."
    ] }, "..."
  ]
}
```

After identifying dependent clauses, the natural language queries could be generated. This process involved hierarchically combining elements from the structured regulation text to provide sufficient context. For instance, in the Norwegian building code TEK17, §12-14 (3) outlines requirements for buildings needing universal design, while its sub-clause (3a) specifies that the width of straight stairs must be at least 1.2 m. Evaluating sub-clause (3a) in isolation could lead to incorrect compliance conclusions if the building being assessed does not, in fact, require universal design as stipulated by its parent clause (3). Thus, to ensure comprehensive understanding, the text from parent paragraphs and letters is appended to each rule, guidance, or pre-accepted solution. Similarly, relevant visual aids that provide additional explanation for certain clauses are also incorporated into the queries. In addition, any identified dependent clauses were appended, forming a complete and context-rich information unit for the LLMs to process during BIM-model preparation and rule execution.

A common part of rule interpretation is classifying regulations in terms of suitability for digitization. However, since the regulations are not digitized in the traditional sense, the categories for classification are instead selected based on how compliance with a given rule should be checked. While this paper primarily details the handling of geometric demands—those requiring the extraction of geometric information from a BIM model—the classification process also identifies other categories, such as subjective requirements, which were considered during the artifact evaluation. This classification is performed through an LLM-based process using prompt engineering to iteratively assess each clause on how it should be checked (Appendix C.1.4).

4.2. Building model preparation

The next processes implemented by the LLM-based ACC artifact are centered around building model preparation. To truly facilitate automation without shifting the manual workload, the approach is to perform the compliance checking without increasing the information requirements for the BIM model, a goal contingent on the availability of sufficiently advanced data extraction functions. To achieve this, the BIM model is not modified during preparation with additional information. Instead, information is automatically extracted in two separate stages as shown in Fig. 3: (1) checking if clauses are applicable to the specific building project; and (2) extracting information from the BIM model needed for rule execution.

Both stages of building model preparation use LLM-based reasoning to generate function calls, which in turn execute predefined IfcOpenShell scripts to extract information from the IFC file. To facilitate this, the LLMs is provided with a list of available tools in a JSON format that describes each tool's purpose and parameters (see example in Appendix C.2.1). To focus the evaluation on the LLM's core capabilities, most of these tools were implemented as placeholders. These are described to the LLMs identically to functional scripts but return predefined

information, allowing us to assess the LLM's ability to select the correct function and parameters, independent of the scripts' full implementation status. For transparency, the appendix provides examples of both a fully implemented script (Appendix C.2.2) and a placeholder (Appendix C.2.3). Appendix C.2.4 shows an example of the LLMs executing tool calls.

In the first stage of building model preparation, each input query—generated during the rule interpretation phase and containing the clause and relevant information—is processed to assess whether it applies to the BIM model or building being checked. This is done by extracting higher level information, such as the building type or intended use of the building (Appendix C.2.5). Furthermore, in the case of TEK17 §12-14 on stairs, it checks whether the building even contains a staircase, and if so, whether it serves multiple dwellings, among other relevant conditions. This mitigates issues during the next stage, in that the artifact does not attempt to extract detailed information where it does not exist or when it is not needed or relevant for rule execution.

In the second stage, the applicable rules are further processed by the LLMs to identify and extract more specific information from the BIM model to facilitate rule execution (Appendix C.2.6). This involves the LLMs interpreting each query, particularly its geometric demands, to determine precisely what data needs to be retrieved from the BIM model. Based on this interpretation, the LLMs then utilizes the function-calling mechanism previously described, selecting appropriate predefined IfcOpenShell scripts and providing the necessary parameters to extract the targeted information (e.g., specific dimensions, properties, or relationships between building elements). As a proof of concept, a function was created that calculates the distance between the start or end of a stair, and the closest door, only given the object id of the stair as input parameters. The script then extracted the geometry and implicitly calculated the start and end position before calculating the distance to all doors. This shows that implicit information such as the location of the start or end of a staircase can be extracted without any pre-processing of the BIM model. This extracted information is then structured and stored for use in the subsequent rule execution phase.

4.3. Rule execution

Once the natural language queries (containing all relevant regulatory information for a given clause) had been generated and the corresponding information had been extracted from the BIM model, the artifact could proceed to rule execution. The core of this stage involved using the LLMs to determine if the building data complied with the specific regulatory clause. By providing both the comprehensive query and the extracted BIM information as distinct 'information packets' to the LLMs, prompt engineering was employed to instruct the model. The conceptual structure for these prompts is shown in Listing 2, while the complete templates are available in (Appendix C.3). Typically, the LLMs was prompted to compare the requirements stated in the query against the provided building data, outputting natural language based compliance reports for each clause.

Listing 2: Conceptual pseudocode for a dynamically generated LLM prompt.

```
messages = [
  {
    "role": "system",
    "content": "<General instructions and system persona>"
  },
  {
    "role": "user",
    "content": "Context: {All relevant regulatory text},
      ↪ {Extracted BIM data}. Task: Check compliance
      ↪ for {specific rule identifier}."
  }
]
```

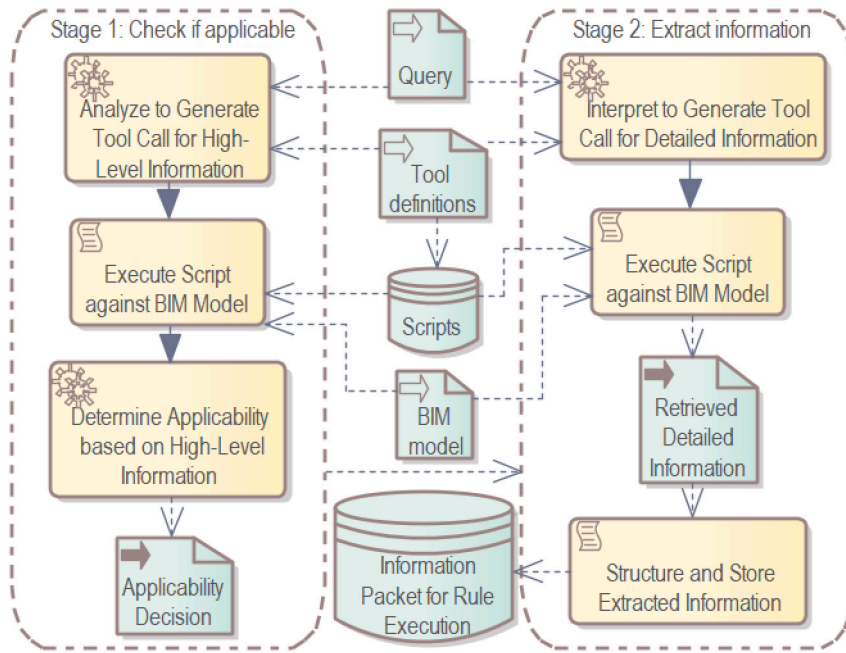



Fig. 3. LLM-driven workflow for BIM model preparation (BPMN 2.0). Notation: gear = Service Task (external service call via API, e.g., LLM); scroll = Script Task (inline code inside the workflow); hollow arrows = inputs; filled arrows = outputs.

4.4. Reporting checking results

The individual natural language compliance reports generated by the LLMs during rule execution – which inherently benefit from the LLM’s ability to produce rich textual explanations – were then further processed by an LLMs for structuring and refinement, and consolidated into a single, comprehensive compliance report for the building. This comprehensive report details for each evaluated clause: the original regulatory clause text, the relevant BIM information extracted for its evaluation, the LLM’s detailed reasoning, a clear compliance status (e.g., ‘Complies,’ ‘Does Not Comply,’ ‘Not Applicable’), and its overall conclusion regarding compliance. This report could then be summarized – again with LLMs assistance – into a more concise compliance summary. The summary aims to outline the general compliance status while specifically highlighting any areas of non-compliance.

5. Artifact evaluation

The artifact was evaluated on each class of functionality to give insights into LLMs ability to facilitate BIM-based ACC. A case study was selected with TEK17 §12-14 (Stairs) (Appendix D) and §12-9 (Bathroom and Toilet) as the regulatory text and a single-family home as the BIM-model. The model contains a total of 206 components, including 41 walls, 20 windows, 11 doors, 2 railings, and 1 stair. An exterior rendering of the model is shown in Fig. 4(a), with an interior view provided in Fig. 4(b). The results from the evaluation were compared to manual analysis to determine accuracy and recall. The evaluation presented in this paper was conducted using the gpt-4o model. Initial prototyping and development of the artifact were performed using gpt-4-turbo, and we transitioned to the newer gpt-4o model for the final evaluation upon its release. To ensure baseline reproducibility, all API calls were made using the provider’s default invocation parameters.

Each sub-process of the artifact was continuously assessed and refined during development. This prompt engineering process was performed exclusively using the development set: TEK17 §12-14 (Stairs), which comprises 87 clauses. To ensure the artifact had not been overfitted to this text and to test its generalization capabilities, the final version was then evaluated on a combined set of 130 clauses. This

evaluation set included the in-domain ‘Stairs’ regulation (66.9% of the set) and the previously unseen, out-of-domain §12-9 (‘Bathroom and Toilet’) regulation (n=43, 33.1%). The evaluation was performed without providing any new tools. The following subsections will evaluate the artifact on each class of functionality.

5.1. Rule interpretation

The first step in evaluating the artifact was to convert the regulation into the specified JSON schema, enabling programmatic validation of the data structure. An excerpt of the resulting JSON is shown in Appendix B.3. Web scraping with HTML parsing was used to automate the conversion of some sections, but structural inconsistencies in the HTML across sections made full automation challenging. Thus, the content still required manual verification. The JSON data included an identifier string which was unique for each distinct clause. These were used to generate dictionaries for storing results from manual and artifact compliance checking steps, so that the results could be compared to calculate accuracy and recall for each step.

To quantitatively evaluate the classification performance, we calculated the precision, recall, and F1 scores. These metrics are derived from the number of True Positives (TP), False Positives (FP), and False Negatives (FN). For the geometric classification task, a TP was a clause correctly identified as geometric, an FP was a non-geometric clause incorrectly labeled as geometric, and an FN was a geometric clause that the artifact failed to identify. The metrics are formally defined in Eqs. (1)–(3):

$$\text{Precision} = \frac{TP}{TP + FP} \quad (1)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2)$$

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3)$$

The dataset used for this evaluation, comprising a development and a test set from different sections of TEK17, is summarized in Table 1.

The proposed artifact was then evaluated on identifying dependencies between clauses. A concrete example of such a dependency is §12-14 (1) (i), which requires stairs without a straight run to be 0.1 m

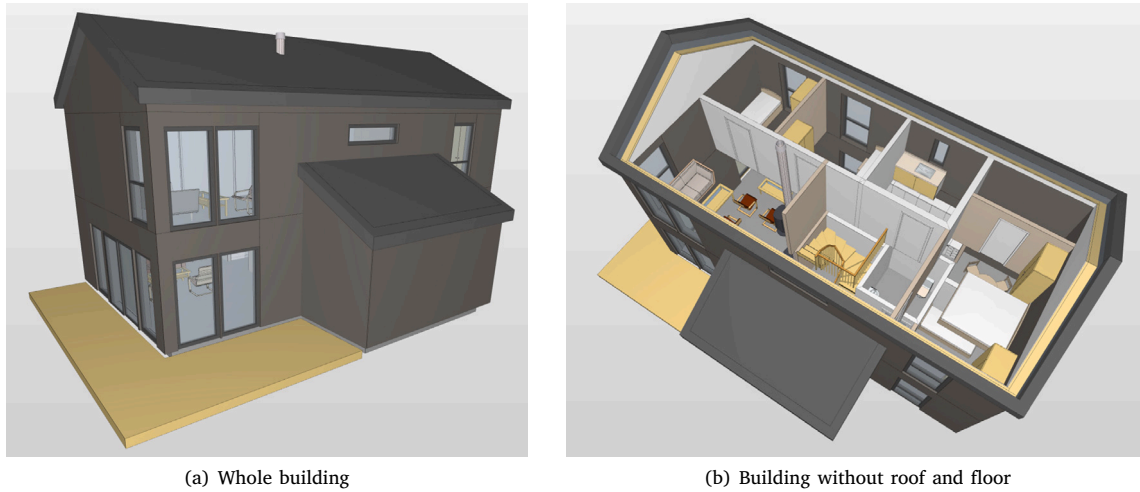


Fig. 4. Single family home case study BIM-model.

Table 1
Summary of the dataset used for evaluation.

Set	Source	Clauses	Dep.	Subj.	Geom.
Development	TEK17 §12-14	87	1	18	46
Test	TEK17 §12-9	43	0	9	18
Total	–	130	1	27	64

Note: Dep. = Dependencies, Subj. = Subjective, Geom. = Geometric.

Table 2
Artifact evaluation results, split by the development set (TEK17 §12-14, n=87) and the unseen test set (TEK17 §12-9, n=43).

Task	Dataset	n	Precision	Recall	F1-Score
Dependency	§12-14	1	1.00	1.00	1.00
	§12-9	0	N/A ^a	N/A ^a	N/A ^a
Geometric	§12-14	46	0.99	0.96	0.97
	§12-9	18	0.95	1.00	0.97
Subjective	§12-14	18	0.95	1.00	0.97
	§12-9	9	0.86	1.00	0.92

^a N/A: Not Applicable. The test set contained no dependent clauses, and the artifact correctly identified all as independent.

wider than the base requirement specified in sub-clause (h). In this case, both clauses must be provided together during compliance checking to determine the actual required dimension. As shown in Table 2, the artifact managed to correctly identify all dependencies while not giving any false positives.

The generated queries were then used to evaluate the artifact's ability to identify geometric and subjective demands. As shown in Table 2, the artifact achieved F1 scores of 0.97 and 0.92 on the test classification tasks.

5.2. Building model preparation

The evaluation of the building model preparation capabilities addressed the two distinct stages: checking clause applicability and the LLM-driven extraction of information from the BIM model. To perform the evaluation, the artifact was tested against TEK17 §12-14 and §12-9 with the single-family home BIM model.

In the first stage, the artifact was able to correctly assess clause applicability given the relevant information. It correctly identified clauses as applicable/not applicable based on high-level model data like building type and the presence of features such as staircases. This initial filtering proved effective in mitigating the LLM's proclivity to comply

with unreasonable requests, in the sense that it did not attempt to extract nonexistent information.

For the second stage, information extraction, the evaluation focused on the LLM's ability to interpret queries (particularly geometric demands), identify the necessary data from the BIM model, and correctly select and execute the appropriate predefined IfcOpenShell scripts via function calls. It is noted that while two scripts were functional, the majority were placeholders (as described in Section 4); this allowed the evaluation to concentrate on the LLM's reasoning and tool-selection capabilities in the context of building model preparation. The artifact correctly specified the required function calls in 60 of the 62 tasks, for a success rate of 96.8%. However, two specific errors were observed in information extraction related to TEK17 §12-14: firstly, the artifact failed to provide the height difference between floors for the pre-accepted solutions §12-14 (1) (e) nr.1 and nr.2, even though this same information was correctly identified and extracted for the parent rule (1) (e) itself. This suggests a potential difficulty for the LLMs in fully propagating information requirements to highly specific sub-conditions. Secondly, when processing §12-14 (1) (i) nr.1, the artifact did not prompt for or provide information indicating whether the stair was internal, a detail pertinent to that clause.

5.3. Rule execution

The evaluation then focused on the artifact's rule execution capabilities. When provided with sufficient information, the artifact's performance was strong; of the 18 clauses that had sufficient data for a complete check, the artifact correctly determined the compliance status with a 100% success rate for that step.

To illustrate the LLM's reasoning process, consider the non-compliant case of §12-14. (1) (a), which requires handrails "on both sides" of a stair. The artifact was provided with the extracted BIM data point: "1 side of the stair has a handrail". The LLM's justification demonstrated a clear logical comparison; it identified the core requirement from the rule ('handrails on both sides'), directly contrasted it with the conflicting fact from the BIM data, and concluded a 'Non-Compliant' verdict. Table 3 qualitatively compares the artifact's summarized rule execution output against a manual check for two example cases. The full, verbatim outputs are available in Appendix E for complete transparency.

This type of transparent, step-by-step reasoning is a key improvement over traditional methods. The accompanying table summarizes this case and one other, comparing the rich output of the LLMs against the more conclusory notes from a manual analysis.

Table 3

Qualitative comparison of outputs for a non-compliant and a compliant clause.

Manual check output	LLM-based ACC output
Case 1: Non-Compliant (§12-14. (1) a) - Handrails on both sides	
“The staircase has safe boundaries on both sides, however the railing is only on one side. The pre-accepted solution is fulfilled by having a railing at the top of the stair where the elevation difference is greater than 0.5 m”.	Verdict: Non-Compliant. Reasoning: The LLMs correctly identifies that the rule requires handrails on “both sides”. It then compares this to the BIM data (“1 side of the stair has a handrail”) and concludes non-compliance, suggesting that a handrail must be added to the other side.
Case 2: Compliant (§12-14. (1) b) - Handrail height	
“The top of the railing is 85.5 cm above the front of the step in accordance with the rule and guidance”.	Verdict: Compliant. Reasoning: The LLMs extracts the required height range (80 cm to 90 cm) from the regulation. It then checks the provided BIM data (85.5 cm) and correctly determines that it falls within the acceptable range, concluding compliance.

Table 4

Comparison of human-generated versus LLM-generated compliance reports (Excerpt for §12-14).

Human baseline	LLM detailed finding
Rule: §12-14. (1) Pre-accepted solution nr.1	
“§12-14. (1) The rule applies to the building. The minimum distance between any doors to the end/start of the staircase is greater than the minimum of 0.5 m according to the pre-accepted solution. Upon initial evaluation the stair looks like a standard and safe staircase”.	“§12-14. (1) Pre-accepted solution nr.1: The building complies with the regulation because the distance between the IfcDoor (id: 0APH1\$gvswMdP6BBXK2k0n) and the start of the internal stair is 1.029 m. This exceeds the minimum required distance of 0.3 m for internal stairs and the general minimum distance of 0.5 m. Therefore, the building meets the specified geometric requirements of the regulation”.
LLM summary conclusion (Overall Section §12-14)	
“Conclusion: The building project generally adheres to most of the geometric and safety regulations regarding stair design, but requires rectification in certain areas. Specifically, the installation of an additional handrail, ensuring the stair meets width requirements, and inclusion of intermediate landings are required to ensure full compliance”.	

5.4. Reporting checking results

The final stage of the artifact’s functionality, reporting the checking results, was then examined. The outputs from the rule execution stage were confirmed to be successfully processed by the artifact to generate the two designed compliance report formats: firstly, a detailed report (Appendix F.1) that accurately consolidated the compliance findings for each rule (including the original regulatory clause, the specific BIM data used for evaluation, the LLM’s reasoning, and the determined compliance status); and secondly, a more concise summary of compliance (Appendix F.2).

To highlight the practical advantages of the proposed method and to verify the output quality, Table 4 presents a comparative analysis of the three report types: the human baseline, the LLM detailed finding, and the LLM overall summary. The comparison demonstrates the LLM’s ability to provide auditable evidence, such as the specific BIM element ID and exact measured values. Furthermore, the overall summary is automatically generated from the collected detailed findings, providing a high-level overview without requiring the additional manual effort that would be necessary to synthesize a similar document.

Table 5

Confusion matrix for the naive LLM baseline on the ‘Stairs’ dataset (n=87).

Actual class	Predicted class			Total
	Compliant	Non-compliant	Not applicable	
Compliant	18	4	2	24
Non-Compliant	0	1	0	1
Not applicable	11	5*	46	62
Total	29	10	48	87

* Denotes the cell where one invalid LLM response was categorized as a misclassification.

Table 6

Confusion matrix for the full artifact on the ‘Stairs’ dataset (n=87).

Actual class	Predicted class			Total
	Compliant	Non-compliant	Not applicable	
Compliant	22	2	0	24
Non-compliant	0	1	0	1
Not applicable	0	0	62	62
Total	22	3	62	87

Table 7

Performance metrics by class, comparing the naive baseline and the full artifact.

Class	Naive baseline			Full artifact		
	P	R	F1	P	R	F1
Compliant	0.62	0.75	0.68	1.00	0.92	0.96
Non-compliant	0.11	1.00	0.20	0.33	1.00	0.50
Not applicable	0.96	0.75	0.84	1.00	1.00	1.00
Macro average	0.56	0.83	0.57	0.78	0.97	0.82
Weighted average	0.86	0.75	0.79	0.99	0.98	0.98

P: Precision, R: Recall, F1: F1-Score.

5.5. Compliance checking performance

To isolate the effect of our engineered framework, we established a naive LLM baseline on the ‘Stairs’ dataset (n=87). While both systems ultimately utilized the same BIM data, the baseline was not tasked with the challenge of dynamic retrieval. Instead, to test the rule execution stage in isolation, the baseline was provided with the correct RAG-assembled clause-level data and building-level facts upfront. It then attempted to classify compliance via a simple direct prompt (Appendix G) in a single turn. In contrast, the full artifact was tasked with the complete reasoning pipeline: it had to autonomously determine what information to retrieve, generate the necessary queries, and process the results through a multi-step workflow. This workflow benefits the rule execution by dividing it into distinct stages, checking applicability first and only proceeding to compliance checking for applicable rules, using complex prompts to generate detailed outputs.

The performance of both approaches is detailed in their respective confusion matrices (Table 5 for the baseline and Table 6 for the full artifact). A direct comparison of the key performance metrics is presented in Table 7, and Fig. 5 clearly visualizes the artifacts improved F1-scores compared to the naive baseline.

The results demonstrate a substantial improvement attributable to our artifact. The full framework shows clear superiority across all categories, increasing the F1-score for the ‘Compliant’ class from 0.68 to 0.96 and achieving a perfect score of 1.00 for the ‘Not Applicable’ class (up from 0.84). Most notably, it more than doubles the F1-score for the challenging ‘Non-Compliant’ class from 0.20 to 0.50. These improvements are explained by (1) a structured multi-step workflow that first obtains and tests applicability facts via tool calls and then retrieves clause-specific BIM before issuing the verdict, and (2) targeted prompt engineering that constrains reasoning and label selection.

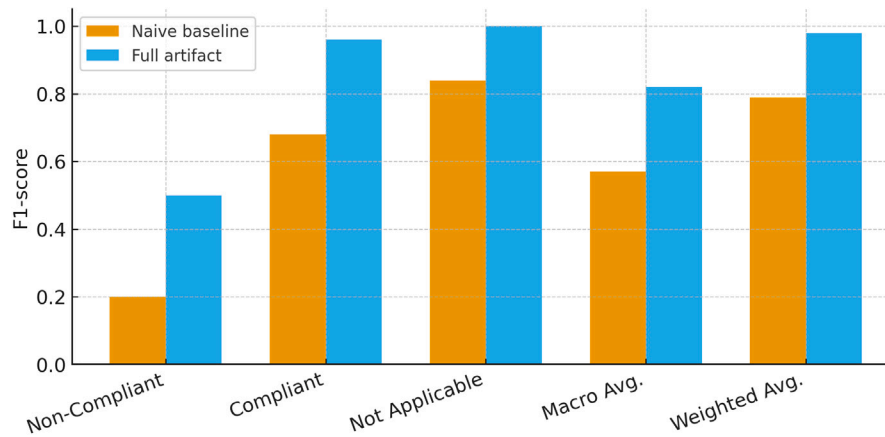


Fig. 5. Class-wise F1-scores for the naive baseline and the full artifact.

6. Discussion

This section will discuss to what extent LLMs can facilitate BIM based ACC, based on the proposed artifact and its evaluation. First, to ensure additions to the knowledge base, the contribution of LLMs for each class of functionality will be discussed, before delving into general discussions of RAG implementation and how the concept of machine interpretability is changing due to LLMs. Finally, possible use cases will be discussed to facilitate application of the artifact in the appropriate environment.

6.1. Rule interpretation

The proposed artifact provides a new approach to rule interpretation within ACC research, and its most significant contribution is demonstrating a path toward eliminating the manual logic rule coding that has traditionally been a major bottleneck in the field. This would resolve what is both a significant challenge for ACC and, as established in the review of related work (Section 2), the prevailing approach in the field: the complex task of converting copious amounts of widely varying regulations into a machine-interpretable format, such as formal logic rules or executable code.

It is important, however, to distinguish this advanced semantic conversion from more fundamental data preparation. While the artifact's LLM-centric approach potentially makes the creation of machine-interpretable logic obsolete, the source text must still satisfy two prerequisites. First, it must be machine-readable—that is, available in a basic digital format that a computer can process at the character level. Second, for the artifact's RAG implementation to function effectively, this machine-readable text must then be specifically pre-processed and structured, in this case by giving it a segmented and hierarchical structure according to the proposed JSON-Schema. However, this pre-processing step is significantly less demanding than the full semantic conversion into logic rules required by traditional approaches.

To better facilitate RAG, regulations should be stored natively in a consistently structured format. All digital formats are machine readable by nature, however to facilitate RAG the format must also be consistently structured. This paper proposes a JSON Schema to validate the data structure in accordance with Norwegian technical requirements from TEK17. Similar schemas could be made for other regulations, allowing the conversion and validation of structured JSON data containing the regulatory text. Other approaches or data formats such as HTML or csv could be considered as long as a consistent structure is obeyed. This way of digitizing regulations requires far less effort than existing work, since no interpretation of clauses or contents are required, only identifying the hierarchical structure is required. The process of identifying the hierarchical structure of regulatory text is

also a suitable target for investigating LLM based automation in future work.

While the evaluation of the artifact provides strong performance with high F1 scores, it is important to acknowledge the limitations regarding the generalizability of these findings. The evaluation was conducted using a single building code, the Norwegian TEK17, and was focused on a relatively simple case study of a single-family home. The linguistic and structural nuances of other international building codes may present different challenges for the LLMs. Furthermore, the results are specific to the GPT-4 series of models; performance could vary with different models or regulations in other languages. While this focused scope was necessary for a proof-of-concept, future work must validate the approach across a wider range of regulations, building typologies, and languages to establish the broader applicability of this LLM-centric methodology.

6.2. Building model preparation

Given a list of predefined scripts, the artifact used LLM based reasoning to determine which scripts should be executed for each given clause, to retrieve relevant information for rule execution. While this shows that LLMs can contribute to building model preparation, it also poses a new problem of creating the predefined scripts.

The evaluation, which utilized placeholder functions, confirms that the LLM can capably identify the correct functions to call based on regulatory queries, but these functions must still be manually implemented. The granularity and reusability of these scripts are also important considerations. The tools developed for this artifact were created specifically for the TEK17 case study, resulting in a mix of both fine-grained and more general-purpose functions. While these scripts are inherently reusable, creating a complete library of them represents a significant development bottleneck. We are confident that a full implementation is feasible, as supported by our successful proof-of-concept script for the challenging task of implicitly identifying a stair's start and end points. A promising direction for mitigating this manual workload is to automate the script-writing process itself. As was observed during development, the LLM could in some cases describe the functionality of a required but missing function. This self-awareness, combined with modern code-generation capabilities, suggests a path toward a system where the LLMs assists in or fully automates the generation of the necessary data extraction scripts, a key area for future investigation.

A potential solution for the first of the two errors observed during information extraction involves creating a system where information retrieved for higher-level rules (e.g., a main clause) is automatically passed down to lower-level rules, such as pre-accepted solutions. This is a feasible approach, as the artifact demonstrated its ability to correctly

extract the relevant information for the parent rule itself. This process of information inheritance follows the same hierarchical principle used during query generation, where higher-level textual context is included to ensure accurate rule interpretation. It is therefore reasonable to extend this principle to also include higher-level extracted geometric data. However, this mechanism has not yet been implemented in the current version of the proposed artifact.

A primary goal during development was to avoid increasing the information requirements for the BIM model. While this was demonstrated to be feasible for geometric information, we acknowledge that the artifact does not address non-geometric design intent. Future work is required to investigate how design intent could be automatically inferred or incorporated. For example, while regulatory requirements like fire-ratings could potentially be mapped to model elements based on the regulatory text, other crucial information, such as the intended use of a room, is more ambiguous and likely still needs to be explicitly modeled by the designer. We maintain that the goal of not simply shifting the manual workload from one task to another is critical when developing any automated process.

6.3. Rule execution

The evaluation showed the artifact could accurately execute rules when provided with sufficient and accurate information from both the regulation and the BIM model. However, a notable limitation in its current implementation is its handling of incomplete BIM information. When data required for a conclusive check is missing, the artifact adheres to a ‘fail-safe’ principle and defaults to a non-compliant verdict, but it does not explicitly flag that the verdict was reached due to data insufficiency. Future work must focus on implementing more robust error-handling. One approach is to add a preliminary checking step or use prompt engineering to instruct the model to return an ‘Insufficient Information’ verdict. A more advanced solution, inspired by related work, would be to transition to a multi-agent framework. This would allow the ‘rule execution’ agent to actively request any missing information from the ‘building model preparation’ agent, resolving the insufficiency directly rather than simply reporting it. Despite this current shortcoming, the results clearly indicate that LLMs can significantly contribute to fully automating the rule execution process.

6.4. Reporting checking results

The artifact was capable of generating both detailed and summarized versions of the compliance reports, thus showing that LLMs can contribute to reporting checking results. This emphasis on detailed reporting contrasts with the general trend identified in the review of related work (Section 2), which found that the reporting of checking results is often not a primary focus in existing ACC research. Thus, existing approaches only provide a brief description of report generation, and the reports are often lacking in detail. The compliance reports provided by the artifact contains both the interpretation of the regulation, the information extracted from the BIM-model and the reasoning done during checking. This gives a comprehensive overview of all aspects of compliance checking in the report. Furthermore, the concise summary also provides a quick overview of the compliance check as a whole. The report also includes recommended actions to fix areas of non-compliance.

6.5. Potential applications

The artifact was developed with DBP as its primary use case. During the official permitting process, a building must comply with all relevant regulations, a requirement which aligns well with the artifact’s iterative and exhaustive checking process. Although not yet a flawless system, it could serve as a valuable support tool, potentially identifying areas of

non-compliance that might be missed during manual checks. Furthermore, the adaptability of LLM-based systems presents an opportunity for municipalities to incorporate their strategic priorities into the ACC process. The LLM could be given special instructions, allowing the system to be easily tailored to local policy goals.

Additionally, the principles of this ACC approach are highly relevant during the building design phase. A key difference from the permitting stage is the need for targeted, on-demand compliance checking. For instance, when an architect is designing a specific component like a staircase railing, the ACC system should ideally check only the rules relevant to that element, not unrelated regulations for bathrooms or HVAC systems. The proposed artifact, with its design for exhaustive checks, is therefore not currently optimized for this targeted design-phase use case. However, its foundational capabilities could still provide value during design, and it could be adapted for this purpose with further development—specifically, by incorporating functionality to filter and apply rules only to selected model components.

7. Conclusions and future work

This paper has presented an artifact developed using DSR principles in order to determine to what extent LLMs can facilitate BIM based ACC. The approach is LLM centric, using LLM reasoning and interpretation as the core of the ACC framework. This is a more holistic approach compared to existing research where LLMs are used to perform or assist digitizing regulations into code or logic rules. In essence, the key innovation is the shift from using LLMs as a tool to aid the digitization process to using them as the core reasoning engine, thereby omitting the need for manually coded logic rules entirely.

The evaluation demonstrated the artifact’s effectiveness across all four classes of ACC functionality. In terms of rule interpretation, the artifact showed that within a suitable framework, an LLM can effectively interpret regulatory content, achieving accuracy and recall comparable to state-of-the-art methods. This indicates that LLMs could eliminate the traditional digitization process by interpreting regulations directly. For building model preparation, the evaluation showed that LLMs can execute predefined scripts according to the information requirements of clauses to extract necessary data from the BIM model. Subsequently, for rule execution, the artifact could accurately determine building compliance, given this information was sufficient. Finally, the reporting of checking results was significantly improved compared to existing approaches, as the LLM could generate more detailed, user-tailored reports that included its reasoning.

The developed artifact serves as a strong proof of concept, demonstrating the significant potential of an LLM-centric approach for BIM-based ACC within the DBP context. For this approach to mature from a research artifact into a production-ready system, however, future research is essential in several key areas. This includes: further automating building model preparation to avoid simply shifting the manual workload from rule interpretation to script programming; broadening the evaluation against a larger selection of clauses and international regulations; improving the artifact’s architecture to better identify and report data insufficiencies; and extending the research to non-geometric and subjective demands to leverage the full adaptability of LLMs.

CRedit authorship contribution statement

Odin Iversen: Writing – original draft, Visualization, Methodology, Investigation, Formal analysis, Conceptualization. **Lizhen Huang:** Writing – review & editing, Supervision, Funding acquisition.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used Gemini and ChatGPT in order to improve grammar, spelling and clarity. After using this tool/service, the authors reviewed and edited the content as needed and takes full responsibility for the content of the publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors acknowledge funding from the CircWOOD project (The Research Council of Norway, project no. 328698), the 1-2-TRESTEG project (The Research Council of Norway, project no. 336488), and NTNU Strategic support for sustainable built environment (Green2050). The authors would also like to express their sincere gratitude to their industry collaborators. We thank Hallstein Søvik, Tor Oskar Ova Johnsen, and Leif Halvor Sunde from Arkitektur for their continuous support and valuable feedback. We are also grateful to Thor Fredrik Eie (Mindshift AS), as well as Ragnhild O. Amdam and Oddvin Farestveit from the Norwegian Directorate for Building Quality (DiBK), for their insightful contributions. The guidance of Professor Christoph Merschbrock and Professor Eilif Hjelseth is also gratefully acknowledged.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.autcon.2025.106707>.

Data availability

The authors do not have permission to share data.

References

- [1] C. Eastman, J.-m. Lee, Y.-s. Jeong, J.-k. Lee, Automatic rule-based checking of building designs, *Autom. Constr.* 18 (8) (2009) 1011–1033, <http://dx.doi.org/10.1016/j.autcon.2009.07.002>.
- [2] F. Noardo, D. Guler, J. Fauth, G. Malacarne, S. Mastrolembro Ventura, M. Azenha, P.-O. Olsson, L. Senger, Unveiling the actual progress of digital building permit: Getting awareness through a critical state of the art review, *Build. Environ.* 213 (2022) 108854, <http://dx.doi.org/10.1016/j.buildenv.2022.108854>.
- [3] T.H. Beach, J.-L. Hippolyte, Y. Rezgui, Towards the adoption of automated regulatory compliance checking in the built environment, *Autom. Constr.* 118 (2020) 103285, <http://dx.doi.org/10.1016/j.autcon.2020.103285>.
- [4] A.R. Hevner, S.T. March, J. Park, S. Ram, Design science in information systems Research1, *MIS Q.* 28 (1) (2004) 75–106, <http://dx.doi.org/10.2307/25148625>.
- [5] S. Malsane, J. Matthews, S. Lockley, P.E.D. Love, D. Greenwood, Development of an object model for automated compliance checking, *Autom. Constr.* 49 (2015) 51–58, <http://dx.doi.org/10.1016/j.autcon.2014.10.004>.
- [6] N.O. Nawari, A generalized adaptive framework (GAF) for automating code compliance checking, *Buildings* 9 (4) (2019) 86, <http://dx.doi.org/10.3390/buildings9040086>.
- [7] F. Li, Y. Song, Y. Shan, Joint extraction of multiple relations and entities from building code clauses, *Appl. Sci.* 10 (20) (2020) 7103, <http://dx.doi.org/10.3390/app10207103>.
- [8] R. Zhang, N. El-Gohary, Hierarchical representation and deep learning-based method for automatically transforming textual building codes into semantic computable requirements, *J. Comput. Civ. Eng.* 36 (5) (2022) 04022022, [http://dx.doi.org/10.1061/\(ASCE\)CP.1943-5487.0001014](http://dx.doi.org/10.1061/(ASCE)CP.1943-5487.0001014).
- [9] X. Xue, J. Zhang, Regulatory information transformation ruleset expansion to support automated building code compliance checking, *Autom. Constr.* 138 (2022) 104230, <http://dx.doi.org/10.1016/j.autcon.2022.104230>.
- [10] O. Doukari, D. Greenwood, K. Rogage, M. Kassem, Object-centred automated compliance checking: A novel, bottom-up approach, *J. Inf. Technol. Constr. ITcon* 27 (17) (2022) 335–362, <http://dx.doi.org/10.36680/j.itcon.2022.017>.
- [11] S. Fuchs, J. Dimyadi, M. Witbrock, R. Amor, Intermediate representations to improve the semantic parsing of building regulations, *Adv. Eng. Inform.* 62 (2024) 102735, <http://dx.doi.org/10.1016/j.aei.2024.102735>.
- [12] P. Zhou, N. El-Gohary, Ontology-based automated information extraction from building energy conservation codes, *Autom. Constr.* 74 (2017) 103–117, <http://dx.doi.org/10.1016/j.autcon.2016.09.004>.
- [13] E. Nuyts, M. Bonduel, R. Verstraeten, Comparative analysis of approaches for automated compliance checking of construction data, *Adv. Eng. Inform.* 60 (2024) 102443, <http://dx.doi.org/10.1016/j.aei.2024.102443>.
- [14] R. Zhang, N. El-Gohary, Clustering-based approach for building code computability analysis, *J. Comput. Civ. Eng.* 35 (6) (2021) 04021021, [http://dx.doi.org/10.1061/\(ASCE\)CP.1943-5487.0000967](http://dx.doi.org/10.1061/(ASCE)CP.1943-5487.0000967).
- [15] K. Kincelova, C. Botton, P. Blanchet, C. Dagenais, Fire safety in tall timber building: A BIM-based automated code-checking approach, *Buildings* 10 (7) (2020) 121, <http://dx.doi.org/10.3390/buildings10070121>.
- [16] Z. Zhang, N. Nisbet, L. Ma, T. Broyd, Capabilities of rule representations for automated compliance checking in healthcare buildings, *Autom. Constr.* 146 (2023) 104688, <http://dx.doi.org/10.1016/j.autcon.2022.104688>.
- [17] P. Patlakas, I. Christovasilis, L. Riparbelli, F.K. Cheung, E. Vakaj, Semantic web-based automated compliance checking with integration of finite element analysis, *Adv. Eng. Inform.* 61 (2024) 102448, <http://dx.doi.org/10.1016/j.aei.2024.102448>.
- [18] J. Zhang, N.M. El-Gohary, Integrating semantic NLP and logic reasoning into a unified system for fully-automated code checking, *Autom. Constr.* 73 (2017) 45–57, <http://dx.doi.org/10.1016/j.autcon.2016.08.027>.
- [19] J. Zhang, N.M. El-Gohary, Semantic NLP-based information extraction from construction regulatory documents for automated compliance checking, *J. Comput. Civ. Eng.* 30 (2) (2016) 04015014, [http://dx.doi.org/10.1061/\(ASCE\)CP.1943-5487.0000346](http://dx.doi.org/10.1061/(ASCE)CP.1943-5487.0000346).
- [20] J. Zhang, A logic-based representation and tree-based visualization method for building regulatory requirements, *Vis. Eng.* 5 (1) (2017) 2, <http://dx.doi.org/10.1186/s40327-017-0043-4>.
- [21] H. Hettiarachchi, A. Dridi, M.M. Gaber, P. Parsafard, N. Bocaneala, K. Breitenfelder, G. Costa, M. Hedblom, M. Juganaru-Mathieu, T. Mechamia, S. Park, H. Tan, A.-R.H. Tawil, E. Vakaj, CODE-ACCORD: A corpus of building regulatory data for rule generation towards automatic compliance checking, *Sci Data* 12 (1) (2025) 170, <http://dx.doi.org/10.1038/s41597-024-04320-x>.
- [22] G. Recski, E. Iklódi, B. Lellmann, Á. Kovács, A. Hanbury, BRISE-plandok: A German legal corpus of building regulations, *Lang Resour. Eval.* 59 (2) (2025) 1043–1082, <http://dx.doi.org/10.1007/s10579-024-09747-7>.
- [23] R. Zhang, N. El-Gohary, Natural language generation and deep learning for intelligent building codes, *Adv. Eng. Inform.* 52 (2022) 101557, <http://dx.doi.org/10.1016/j.aei.2022.101557>.
- [24] X. Xue, J. Zhang, Building codes part-of-speech tagging performance improvement by error-driven transformational rules, *J. Comput. Civ. Eng.* 34 (5) (2020) 04020035, [http://dx.doi.org/10.1061/\(ASCE\)CP.1943-5487.0000917](http://dx.doi.org/10.1061/(ASCE)CP.1943-5487.0000917).
- [25] X. Xue, J. Zhang, Part-of-speech tagging of building codes empowered by deep learning and transformational rules, *Adv. Eng. Inform.* 47 (2021) 101235, <http://dx.doi.org/10.1016/j.aei.2020.101235>.
- [26] J.-K. Lee, K. Cho, H. Choi, S. Kim, S.H. Cha, High-level implementable methods for automated building code compliance checking, *Dev. Built Environ.* 15 (2023) 100174, <http://dx.doi.org/10.1016/j.dibe.2023.100174>.
- [27] L. Jiang, J. Shi, C. Wang, Multi-ontology fusion and rule development to facilitate automated code compliance checking using BIM and rule-based reasoning, *Adv. Eng. Inform.* 51 (2022) 101449, <http://dx.doi.org/10.1016/j.aei.2021.101449>.
- [28] Y. Li, M. Yang, Q. Zhao, Z. Li, Z. Ma, Y. Liu, X. Hei, A semantic representation method of building codes applied to compliance checking, *Mathematics* 11 (11) (2023) 2552, <http://dx.doi.org/10.3390/math11112552>.
- [29] J. Wu, X. Xue, J. Zhang, Invariant signature, logic reasoning, and semantic natural language processing (NLP)-based automated building code compliance checking (i-SNACC) framework, *J. Inf. Technol. Constr. ITcon* 28 (1) (2023) 1–18, <http://dx.doi.org/10.36680/j.itcon.2023.001>.
- [30] S. Li, J. Wang, Z. Xu, Automated compliance checking for BIM models based on Chinese-NLP and knowledge graph: An integrative conceptual framework, *Eng. Constr. Archit. Manag.* 32 (6) (2024) 3832–3856, <http://dx.doi.org/10.1108/ECAM-10-2023-1037>.
- [31] L. Jiang, J. Shi, Z. Pan, C. Wang, N. Mulatibieke, A multiscale modelling approach to support knowledge representation of building codes, *Buildings* 12 (10) (2022) 1638, <http://dx.doi.org/10.3390/buildings12101638>.
- [32] R. Kruiper, B. Kumar, R. Watson, F. Sadeghineko, A. Gray, I. Konstas, A platform-based natural language processing-driven strategy for digitalising regulatory compliance processes for the built environment, *Adv. Eng. Inform.* 62 (2024) 102653, <http://dx.doi.org/10.1016/j.aei.2024.102653>.
- [33] J. Zhang, N.M. El-Gohary, Extending building information models semiautomatically using semantic natural language processing techniques, *J. Comput. Civ. Eng.* 30 (5) (2016) C4016004, [http://dx.doi.org/10.1061/\(ASCE\)CP.1943-5487.0000536](http://dx.doi.org/10.1061/(ASCE)CP.1943-5487.0000536).
- [34] R. Zhang, N. El-Gohary, Transformer-based approach for automated context-aware IFC-regulation semantic information alignment, *Autom. Constr.* 145 (2023) 104540, <http://dx.doi.org/10.1016/j.autcon.2022.104540>.
- [35] J. Wu, J. Zhang, Model validation using invariant signatures and logic-based inference for automated building code compliance checking, *J. Comput. Civ. Eng.* 36 (3) (2022) 04022002, [http://dx.doi.org/10.1061/\(ASCE\)CP.1943-5487.0001002](http://dx.doi.org/10.1061/(ASCE)CP.1943-5487.0001002).
- [36] T. Bloch, R. Sacks, Clustering information types for semantic enrichment of building information models to support automated code compliance checking, *J. Comput. Civ. Eng.* 34 (6) (2020) 04020040, [http://dx.doi.org/10.1061/\(ASCE\)CP.1943-5487.0000922](http://dx.doi.org/10.1061/(ASCE)CP.1943-5487.0000922).

- [37] A. Karmakar, V.S.K. Delhi, Restructuring elements in IFC topology through semantic enrichment: A case of automated compliance checking, in: ISARC Proceedings 2023 Proceedings of the 40th ISARC, Chennai, India, 2023, pp. 458–465, <http://dx.doi.org/10.22260/ISARC2023/0062>.
- [38] N.O. Nawari, Generalized adaptive framework for computerizing the building design review process, *J. Arch. Eng.* 26 (1) (2020) 04019026, [http://dx.doi.org/10.1061/\(ASCE\)AE.1943-5568.0000382](http://dx.doi.org/10.1061/(ASCE)AE.1943-5568.0000382).
- [39] S. Fischer, H. Urban, C. Schranz, P. Loibl, L. van Berlo, Extending information delivery specifications for digital building permit requirements, *Dev. Built Environ.* 20 (2024) 100560, <http://dx.doi.org/10.1016/j.dibe.2024.100560>.
- [40] J. Zheng, M. Fischer, Dynamic prompt-based virtual assistant framework for BIM information search, *Autom. Constr.* 155 (2023) 105067, <http://dx.doi.org/10.1016/j.autcon.2023.105067>.
- [41] D. Fernandes, S. Garg, M. Nikkel, G. Guven, A GPT-powered assistant for real-time interaction with building information models, *Buildings* 14 (8) (2024) 2499, <http://dx.doi.org/10.3390/buildings14082499>.
- [42] S. Jeongmin, S. Sangyun, Implementation of an automated code checking algorithm based on site analysis, *Buildings* 14 (6) (2024) 1654, <http://dx.doi.org/10.3390/buildings14061654>.
- [43] X. Li, S. Zhao, Y. Shen, Y. Xue, T. Li, H. Zhu, Big data-driven TBM tunnel intelligent construction system with automated-compliance-checking (ACC) optimization, *Expert Syst. Appl.* 244 (2024) 122972, <http://dx.doi.org/10.1016/j.eswa.2023.122972>.
- [44] J.P. Martins, A. Monteiro, LicA: A BIM based automated code-checking application for water distribution systems, *Autom. Constr.* 29 (2013) 12–23, <http://dx.doi.org/10.1016/j.autcon.2012.08.008>.
- [45] S. Macit Ilal, H.M. Günaydin, Computer representation of building codes for automated compliance checking, *Autom. Constr.* 82 (2017) 43–58, <http://dx.doi.org/10.1016/j.autcon.2017.06.018>.
- [46] M. Aydın, Building information modeling based automated building regulation compliance checking Asp.net web software, *IASC* 28 (1) (2021) 11–25, <http://dx.doi.org/10.32604/iasc.2021.015065>.
- [47] S. Fuchs, M. Witbrock, J. Dimyadi, R. Amor, Using large language models for the interpretation of building regulations, *EPPM-J.* 14 (4) (2024) 0035, <http://dx.doi.org/10.32738/JEPPM-2024-0035>.
- [48] D. Al-Turki, H. Hettiarachchi, M. Medhat Gaber, M.M. Abdelsamea, S. Basurra, S. Iranmanesh, H. Saadany, E. Vakaj, Human-in-the-loop learning with LLMs for efficient RASE tagging in building compliance regulations, *IEEE Access* 12 (2024) 185291–185306, <http://dx.doi.org/10.1109/ACCESS.2024.3512434>.
- [49] F. Yang, J. Zhang, Prompt-based automation of building code information transformation for compliance checking, *Autom. Constr.* 168 (2024) 105817, <http://dx.doi.org/10.1016/j.autcon.2024.105817>.
- [50] N. Chen, X. Lin, H. Jiang, Y. An, Automated building information modeling compliance check through a large language model combined with deep learning and ontology, *Buildings* 14 (7) (2024) 1983, <http://dx.doi.org/10.3390/buildings14071983>.
- [51] H. Ying, R. Sacks, From automatic to autonomous: A large language model-driven approach for generic building compliance checking, in: *Proc. 41st Int. Conf. CIB W78, Marrakech, Morocco, (ISSN: 2706-6568) 2024*, pp. w78–2024–59, <http://itc.scix.net/paper/w78-2024-59>.
- [52] I. Kim, J. Choi, E.A.L. Teo, H. Sun, Development of K-BIM e-submission prototypical system for the openBIM-based building permit framework, *J. Civ. Eng. Manag.* 26 (8) (2020) 744–756, <http://dx.doi.org/10.3846/jcem.2020.13756>.
- [53] Y. Demir Altıntaş, M.E. Ilal, Loose coupling of GIS and BIM data models for automated compliance checking against zoning codes, *Autom. Constr.* 128 (2021) 103743, <http://dx.doi.org/10.1016/j.autcon.2021.103743>.
- [54] W. Fan, Y. Ding, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua, Q. Li, A survey on RAG meeting LLMs: Towards retrieval-augmented large language models, in: *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Min. KDD '24*, Association for Computing Machinery, New York, NY, USA, 2024, pp. 6491–6501, <http://dx.doi.org/10.1145/3637528.3671470>.
- [55] M. Plonka, K. Kocot, K. Holda, K. Daniec, A. Nawrat, A comparative evaluation of the effectiveness of document splitters for large language models in legal contexts, *Expert Syst. Appl.* 272 (2025) 126711, <http://dx.doi.org/10.1016/j.eswa.2025.126711>.
- [56] H. Ma, Y. Lu, Z. Xiao, J. Feng, H. Zhang, J. Yu, SDD-lawllm: Advancing intelligent legal systems through synthetic data-driven fine-tuning of large language models, *Electronics* 14 (4) (2025) 742, <http://dx.doi.org/10.3390/electronics14040742>.
- [57] J. Huang, M. Wang, Y. Cui, J. Liu, L. Chen, T. Wang, H. Li, J. Wu, Layered query retrieval: An adaptive framework for retrieval-augmented generation in complex question answering for large language models, *Appl. Sci.* 14 (23) (2024) 11014, <http://dx.doi.org/10.3390/app142311014>.
- [58] S.V.-T. Tran, J. Yang, R. Hussain, N. Khan, E.C. Kimito, A. Pedro, M. Sotani, U.-K. Lee, C. Park, Leveraging large language models for enhanced construction safety regulation extraction, *J. Inf. Technol. Constr. ITcon* 29 (45) (2024) 1026–1038, <http://dx.doi.org/10.36680/j.itcon.2024.045>.
- [59] A. Kalyuzhnaya, S. Mityagin, E. Lutsenko, A. Getmanov, Y. Aksenkin, K. Fatkhiev, K. Fedorin, N.O. Nikitin, N. Chichkova, V. Vorona, A. Boukhanovsky, LLM agents for smart city management: Enhancing decision support through multi-agent AI systems, *Smart Cities* 8 (1) (2025) 19, <http://dx.doi.org/10.3390/smartcities8010019>.
- [60] R.S.M. Wahidur, S. Kim, H. Choi, D.S. Bhatti, H.-N. Lee, Legal query RAG, *IEEE Access* 13 (2025) 36978–36994, <http://dx.doi.org/10.1109/ACCESS.2025.3542125>.
- [61] L. Giray, Prompt engineering with ChatGPT: A guide for academic writers, *Ann. Biomed. Eng.* 51 (12) (2023) 2629–2633, <http://dx.doi.org/10.1007/s10439-023-03272-4>.
- [62] K. Hebenstreit, R. Praas, L.P. Kiesewetter, M. Samwald, An automatically discovered chain-of-thought prompt generalizes to novel models and datasets, 2023, <http://dx.doi.org/10.48550/arXiv.2305.02897>, [arXiv:2305.02897](https://arxiv.org/abs/2305.02897).
- [63] S.R. Moghaddam, C.J. Honey, Boosting theory-of-mind performance in large language models via prompting, 2023, <http://dx.doi.org/10.48550/arXiv.2304.11490>, [arXiv:2304.11490](https://arxiv.org/abs/2304.11490).
- [64] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, S. Yao, Reflexion: Language agents with verbal reinforcement learning, 2023, <http://dx.doi.org/10.48550/arXiv.2303.11366>, [arXiv:2303.11366](https://arxiv.org/abs/2303.11366).
- [65] V. Nair, E. Schumacher, G. Tso, A. Kannan, DERA: Enhancing large language model completions with dialog-enabled resolving agents, in: T. Naumann, A. Ben Abacha, S. Bethard, K. Roberts, D. Bitterman (Eds.), *Proc. 6th Clin. Nat. Lang. Process. Workshop*, Association for Computational Linguistics, Mexico City, Mexico, 2024, pp. 122–161, <http://dx.doi.org/10.18653/v1/2024.clinicalnlp-1.12>.